

KIALI

Reporte Técnico de Arquitectura y Diseño de Software
«Tecnología que cuida sin que lo notes.»

Asignatura:	Cómputo Móvil
Grupo:	03
Semestre:	2026-2
Fecha:	29 de mayo de 2026
Profesor:	Ing. Marduk Pérez de Lara Domínguez
Equipo:	Swifties
Integrantes:	Sánchez Medina José Santiago Torres Pimentel Obed
Repositorio:	github.com/obeedt/AppSwift
Versión del documento:	1.0

1. Descripción Ejecutiva del Proyecto

Kiali es una aplicación móvil nativa para iOS diseñada para reducir la brecha digital que afecta a los adultos mayores en México y América Latina. Su nombre proviene del náhuatl y significa cuidar, principio que articula cada decisión de diseño y arquitectura. El proyecto nació durante el Hackathon Swift Changemakers 2026, organizado por iOS Dev Lab de la Facultad de Ingeniería de la UNAM bajo el mandato de construir soluciones iOS que implementen Inteligencia Artificial Centrada en el Humano (HCAI).

La premisa arquitectónica central de Kiali es la IA invisible: la inteligencia artificial no se manifiesta como chatbot ni como interfaz explícita que el usuario deba reconocer como «tecnología de IA». Opera como una capa de comportamiento proactivo que anticipa necesidades, genera intervenciones contextuales y guía al usuario a través de tareas del teléfono en pasos simples, sin que el adulto mayor perciba que interactúa con un sistema inteligente. Esta decisión responde directamente a la investigación de MICARE Chile (2025), que documenta que las interfaces tecnológicamente explícitas generan ansiedad y rechazo en la población objetivo.

La aplicación se estructura en tres módulos principales dentro de un TabView: Asistente, que provee guía paso a paso e intervenciones proactivas; Rutinas, que gestiona recordatorios de medicamentos, citas y actividades; y Perfil, que centraliza la ficha médica y la red de contactos de emergencia. El prototipo funcional fue construido en Swift 5.9 con SwiftUI y arquitectura MVVM en 8 horas, y validado en simulador Xcode 16 sobre iPhone SE (3.ª generación) e iPhone 15.

2. Requerimientos de la Aplicación

El levantamiento de requerimientos partió de un análisis doble: la investigación bibliográfica sobre brecha digital en adultos mayores en México (INEGI ENDUTIH, 2022; MICARE Chile, 2025) y las restricciones del hackathon, que obligaron a priorizar con rigor cada funcionalidad. El resultado se organiza en reglas de negocio, requerimientos funcionales y requerimientos no funcionales.

2.1 Reglas de Negocio

Las reglas de negocio son las restricciones invariantes del sistema que ninguna decisión técnica puede vulnerar. Kiali define diez reglas fundamentales. La aplicación debe operar completamente offline para las funciones de Rutinas y Perfil (RN-01); ningún dato personal o médico puede transmitirse a servidores externos sin consentimiento explícito (RN-02); el usuario mayor no debe crear cuenta ni autenticarse —la configuración inicial puede completarla un familiar— (RN-03). Las rutinas eliminadas no son recuperables en el MVP (RN-04). Una rutina es mutuamente excluyente en su tipo: fecha específica o recurrente, nunca ambas (RN-05). El Asistente muestra como máximo una tarjeta de intervención proactiva activa a la vez (RN-06). El Contacto de Alertas recibe notificación automática si una rutina crítica no se confirma en el margen de tiempo configurado (RN-07). Todos los elementos táctiles deben cumplir el mínimo de 44×44 puntos de Apple HIG y superar el ratio de contraste WCAG AA de 4.5:1 (RN-08). La app no reproduce sonido ni vibración sin habilitación explícita del usuario en los ajustes (RN-09). En el MVP, los datos se almacenan en memoria; la persistencia real con SwiftData es bloqueante para la distribución en App Store (RN-10).

2.2 Requerimientos Funcionales

Los requerimientos funcionales se clasifican por módulo y se priorizan bajo el esquema MoSCoW. En el módulo Asistente, los requerimientos Must incluyen: saludo contextual personalizado (RF-A01), cuadrícula de 8 tareas frecuentes con SF Symbols (RF-A02), campo de consulta en texto libre (RF-A03), despliegue de guía de pasos numerados (RF-A04), tarjeta de intervención proactiva al detectar olvido (RF-A05), y confirmación de la tarjeta con «Ya lo hice» (RF-A06). Como Should se incluye la animación de respiración del logo (RF-A07) y el botón de demostración «Simular olvido» (RF-A08). La síntesis de voz TTS es un Should implementado parcialmente —el toggle existe pero la integración de AVSpeechSynthesizer queda para v2.0— (RF-A09). El reconocimiento de voz STT es un Could fuera del MVP (RF-A10).

En el módulo Rutinas, los requerimientos Must abarcan: lista de rutinas del día separada en Pendientes y Completadas (RF-R01), encabezado con fecha y conteo de pendientes (RF-R02), filtrado por categoría mediante chips (RF-R03), creación de rutina vía sheet modal (RF-R04), soporte de tipo fecha específica con DatePicker gráfico y tipo recurrente con chips LMMJVSD (RF-R05), swipe-to-delete (RF-R06) y toggle de completada con animación (RF-R08). La edición de rutina existente con el sheet pre-rellenado es un Should implementado (RF-R07). Las notificaciones locales mediante UNUserNotificationCenter son un Must diferido a v2.0 (RF-R09). En el módulo Perfil, los Must incluyen: avatar con nombre y edad (RF-P01), ficha médica editable (RF-P02), Contacto de Alertas con estado de vinculación (RF-P04), sección de Emergencias con acceso al 911 (RF-P05), Red de Apoyo con llamada directa (RF-P06). Los controles de accesibilidad del Asistente —toggle TTS, sliders de volumen y velocidad— son Should implementados (RF-P07, RF-P08).

2.3 Requerimientos No Funcionales

Los requerimientos no funcionales son especialmente críticos en Kiali dado que la población objetivo incluye usuarios con posibles limitaciones visuales, motrices o cognitivas. En rendimiento, el arranque de la pantalla de Asistente debe completarse en menos de 2 segundos en un iPhone SE 3.^a generación (RNF-01), el bundle de la app no debe superar 50 MB (RNF-10). En accesibilidad, todos los elementos táctiles deben tener área mínima de 44×44 puntos (RNF-02), la tipografía debe escalar correctamente con Dynamic Type hasta xxxLarge (RNF-03, parcialmente cumplido), y el contraste de color debe superar 4.5:1 WCAG AA (RNF-05). El deployment target es iOS 16.0 (RNF-04). La app soporta únicamente orientación Portrait (RNF-06). La arquitectura MVVM debe mantenerse con separación de capas en archivos independientes (RNF-07). Toda información personal y médica permanece on-device, con cero llamadas a servidores externos en el MVP (RNF-08). La interfaz está completamente en español mexicano con arquitectura preparada para i18n (RNF-09).

3. Viabilidad Técnica

Una pregunta recurrente al revisar el diseño de Kiali es si sus promesas —en particular la de una IA que opera de forma invisible y proactiva— son técnicamente viables en el ecosistema iOS. La respuesta requiere distinguir con precisión entre lo que el MVP implementa hoy, lo que es alcanzable a corto plazo con frameworks nativos de Apple, y lo que presenta limitaciones reales que deben comunicarse con honestidad.

3.1 La IA Invisible: Cómo Funciona Realmente

El motor de intervención proactiva del MVP no depende de un modelo de lenguaje ni de APIs externas. Es lógica de negocio local: GestorRutinas evalúa si existe alguna rutina activa cuya hora programada ya pasó y cuya propiedad completada permanece en false, y de ser así establece mostrarIntervencion = true. Esta lógica es deliberadamente simple, completamente viable en Swift, y respeta la privacidad del usuario dado que no transmite dato alguno fuera del

dispositivo.

La evolución hacia una IA genuinamente personalizada descansa en Core ML, el framework de inferencia on-device de Apple que permite importar modelos entrenados con Create ML y ejecutarlos sin conectividad. Modelos de clasificación sobre datos tabulares —frecuencia de olvido, horarios preferidos, patrones de confirmación— pueden entrenarse sobre el historial local del usuario y exportarse como archivos .mlmodel integrados directamente en la app. Esta ruta es técnicamente viable sin backend y sin costos de API, aunque requiere datos históricos suficientes, lo que la sitúa en v3.0 o posterior.

3.2 Funcionalidades Viables a Corto Plazo

Las siguientes capacidades son alcanzables sin infraestructura de backend y tienen precedente documentado en las APIs de Apple. Las notificaciones locales de rutinas utilizan `UNUserNotificationCenter` con un `UNCalendarNotificationTrigger` que se registra al crear cada rutina y se cancela al marcarla como completada; la complejidad es baja. La síntesis de voz TTS con `AVSpeechSynthesizer` soporta voces de alta calidad en español mexicano (es-MX) desde iOS 16, opera completamente offline y sus parámetros de velocidad y volumen mapean directamente a los sliders ya presentes en el módulo Perfil. La persistencia real con `SwiftData`, disponible desde iOS 17, permite decorar el struct `Recordatorio` con el macro `@Model` y obtener almacenamiento automático en SQLite sin código de mapeo adicional. El escaneo de recetas médicas se implementa mediante `DataScannerViewController` de `VisionKit`, que provee OCR on-device en tiempo real, también sin transmitir datos a ningún servidor. Las llamadas directas desde Perfil utilizan el URL scheme `tel://` nativo, cuya implementación es de una sola línea.

3.3 Limitaciones Reales

La IA conversacional con comprensión de lenguaje natural de amplio espectro —comparable a GPT-4 o Gemini— requiere llamadas a APIs externas con latencia de red, costos por token y políticas de privacidad específicas para datos médicos sensibles. Apple Intelligence no estaba disponible en español en la fecha del hackathon, limitando las opciones on-device a pattern matching y modelos de clasificación livianos. La alternativa viable para el MVP es un sistema de palabras clave con respuestas predefinidas que cubre el 80% de los casos de uso documentados. La sincronización familiar en tiempo real requiere un canal push server-to-device que implica backend, almacenamiento de tokens de dispositivo y manejo de sesiones, lo que supera el alcance del MVP pero es viable en v2.0 con Firebase Cloud Messaging como plataforma serverless.

4. Alcance y Definición del MVP

El Producto Mínimo Viable de Kiali representa la intersección entre la visión completa del producto y las restricciones del hackathon: 8 horas de desarrollo y un equipo de dos personas. La estrategia de definición del MVP siguió un principio de reducción por capas: primero se identificó la propuesta de valor nuclear —hacer que un adulto mayor pueda gestionar sus medicamentos y recibir guía de uso del teléfono sin sentirse perdido— y a partir de ahí se incorporó funcionalidad solo cuando el valor era claro y la implementación viable en el tiempo disponible.

El MVP entrega tres módulos funcionales conectados por un `TabView`. El Asistente muestra saludo contextual, cuadrícula de 8 tareas, campo de consulta y tarjeta de intervención proactiva con su lógica de estado. Rutinas soporta creación, edición, filtrado, completado y eliminación de rutinas para los dos tipos (fecha específica y recurrente), incluyendo el selector de calendario gráfico y los chips de días de la semana. Perfil presenta la ficha médica, el sistema de contactos y los controles de accesibilidad. Todo opera con datos en memoria durante la sesión activa.

Lo que el MVP excluye deliberadamente: notificaciones locales (sin alertas en la hora programada), síntesis de voz funcional (toggle presente, AVSpeechSynthesizer no integrado), reconocimiento de voz, persistencia entre sesiones, autenticación y comunicación con servidores externos. La hoja de ruta define cuatro versiones: v1.0 es el MVP del hackathon; v2.0 añade SwiftData, UserNotifications, TTS real y llamadas directas; v3.0 incorpora CloudKit, VisionKit y dashboard familiar; v4.0 integra App Intents para Siri, watchOS companion app y Core ML para personalización.

El proceso de definición del alcance enseñó al equipo una lección fundamental de gestión de producto: la funcionalidad más difícil de eliminar del backlog no es la técnicamente compleja, sino la que parece trivial de implementar pero que consume tiempo desproporcionado en pruebas y ajuste de experiencia de usuario. Las notificaciones locales ilustran este punto: una llamada a `UNUserNotificationCenter` es de baja complejidad de código, pero requiere gestión del ciclo de vida de permisos, prueba obligatoria en dispositivo real —el simulador de Xcode no dispara notificaciones de manera confiable— y manejo del edge case donde el usuario edita una rutina cuya notificación ya estaba programada y debe cancelarse y reprogramarse. Priorizar este elemento como MVP+1 liberó tiempo para pulir la experiencia central de los tres módulos.

El valor del prototipo reside en validar la arquitectura y la propuesta de valor, no en entregar un producto listo para producción: Kiali demuestra que la IA invisible es implementable en iOS con arquitectura MVVM y diseño accesible para adultos mayores, todo dentro del ecosistema SwiftUI sin dependencias de terceros.

5. Dispositivos Objetivo

Kiali está diseñada para iPhone a partir del iPhone SE (3.^a generación) con iOS 16.0 o superior, en orientación Portrait exclusivamente. El iPhone SE es el punto de entrada mínimo por su precio accesible —el modelo que los hijos adquieren habitualmente para sus padres como primera experiencia iOS— y su capacidad técnica suficiente para ejecutar SwiftUI con fluidez. La pantalla de 4.7 pulgadas con resolución de 375×667 puntos es el caso límite de diseño: la cuadrícula de 8 tareas del Asistente y los chips de categoría en Rutinas deben ser cómodos a ese tamaño sin comprimir las áreas táctiles por debajo de los 44pt requeridos.

Los modelos de referencia primaria del diseño son iPhone 13 y 14 (390×844 pt, 6.1 pulgadas), en los que todos los elementos tienen espacio generoso. Los modelos de pantalla grande —iPhone 14 Plus, 15 Pro Max— benefician especialmente a usuarios con baja agudeza visual, dado que el espacio adicional permite que el texto escale con Dynamic Type sin que los layouts se rompan. La orientación Landscape queda fuera del alcance del MVP de manera deliberada: los adultos mayores tienden a sostener el teléfono en posición vertical de manera natural, y soportar ambas orientaciones duplicaría el trabajo de adaptación de layouts sin un beneficio proporcional. Esta restricción se declara en el `Info.plist` limitando `UISupportedInterfaceOrientations` a `UIInterfaceOrientationPortrait`.

El soporte de Dynamic Type merece atención especial en el contexto de Kiali. iOS permite al usuario configurar tamaños de texto desde Extra Small hasta AX5 (Accessibility Extra Extra Extra Extra Large), donde el texto base puede crecer hasta cuatro veces su tamaño original. Las vistas de Kiali implementadas con SwiftUI usando modificadores `.font(.body)`, `.font(.headline)` y similares escalan automáticamente, pero ciertos elementos con tamaños fijos en puntos —como los íconos de 28pt en la cuadrícula del Asistente o las celdas de 72pt en la lista de Rutinas— requieren revisión explícita con tamaños AX grandes para garantizar que el texto no se desborde de sus contenedores. Esta es una de las áreas marcadas como parcialmente cumplida en el requerimiento RNF-03 y representa trabajo pendiente para v2.0.

6. Wireframes y Documentación de Pantallas

Esta sección documenta las diez pantallas principales de Kiali desde la perspectiva de arquitectura de información: qué datos presenta cada vista, cuál es su tipo y vigencia, de dónde provienen, qué operaciones expone al usuario y qué transiciones de navegación dispara.

Pantalla 1 — Asistente: Vista Principal (AsistenteView)

Esta es la pantalla raíz de la pestaña Asistente y la primera que ve el usuario al abrir la app. Se divide en cinco zonas verticales. La superior muestra el saludo contextual —«Buenos días, Daniel»— construido en tiempo de ejecución concatenando el nombre del perfil con la categoría horaria calculada a partir de `Calendar.current.component(.hour)`. Debajo, la animación `SubtleLogoAnimation` ejecuta un ciclo de escala entre 0.97 y 1.0 con `withAnimation(.easeInOut(duration:2).repeatForever)` para comunicar que el sistema está activo. La cuadrícula de ocho tareas frecuentes se implementa con `LazyVGrid` de dos columnas; cada celda muestra un SF Symbol de 28pt y una etiqueta de 13pt. Las tareas corresponden a las acciones que los adultos mayores reportan como más frecuentes y más difíciles (MICARE Chile, 2025): WhatsApp, Llamar, Ver fotos, Volumen, Wi-Fi, Alarma, Tomar foto y Mi ubicación. El campo de texto inferior actualiza `@State` var `consultaActual`, que activa la `TarjetaGuia` con pasos numerados. Todos los datos estáticos de tareas y pasos residen en `AppTheme.swift`; su vigencia es permanente en el MVP. Al seleccionar una tarea o escribir una consulta, el Asistente despliega inline una `TarjetaGuia` con entre tres y seis pasos en lenguaje llano, cuyos textos son strings estáticos en un diccionario `[String: [String]]` en `AppTheme.swift`; la operación es exclusivamente de lectura.

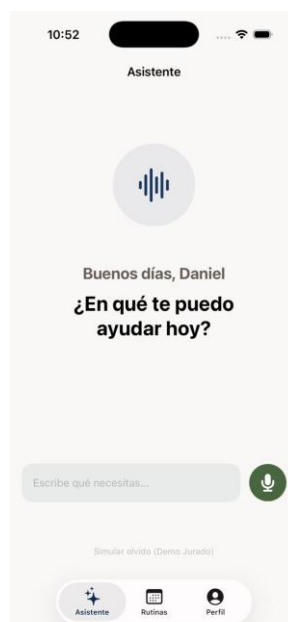


Figura 1. Pantalla 1 — Asistente: Vista Principal. Saludo contextual personalizado y cuadrícula de 8 tareas frecuentes.

Pantalla 2 — Asistente: Tarjeta de Intervención Proactiva

La TarjetaIntervencion se superpone como overlay sobre la vista del Asistente. Muestra el ícono de la categoría de la rutina, la hora programada, el nombre y el detalle de la misma. El botón «Ya lo hice» tiene fondo alertaRoja (#B80000) y altura de 56pt —el elemento táctil más grande de la pantalla de manera intencional—. La información proviene del objeto Recordatorio accedido a través del @ObservedObject gestorRutinas. La confirmación ejecuta gestorRutinas.confirmarIntervencion(), que actualiza completada = true en la rutina asociada y establece mostrarIntervencion = false.

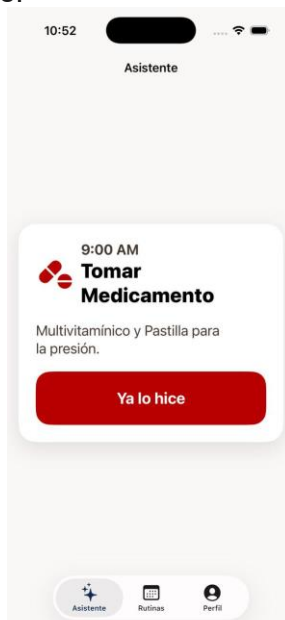


Figura 2. Pantalla 3 — Asistente: Tarjeta de Intervención Proactiva. Botón «Ya lo hice» en rojo con 56pt de alto.

Pantalla 3 — Rutinas: Lista Principal (RecordatoriosView)

Encabezado con la fecha en formato largo («Lunes 20 de Abril») y el contador dinámico «Tienes N pendientes» obtenido de gestorRutinas.pendientesHoy.count. Debajo, una fila horizontal scrollable de chips de filtro (Todos, Medicina, Cita médica, Ejercicio, Comida, General) gestiona el estado @State var categoriaSeleccionada. La lista se divide en secciones Pendientes y Completadas. Cada RowRutina muestra ícono de categoría, nombre, detalle, hora en rojo y un círculo de estado. El tap en el círculo invoca toggleCompletada(id:), que anima el ítem a la sección Completadas. El swipe-to-delete conecta con .onDelete y llama a eliminarRutinas(offsets:). El botón flotante «+ Agregar» presenta el EditorRutinaSheet mediante .sheet(isPresented:).



Figura 3a. Pantalla 4 — Rutinas: 3 tareas pendientes del día.

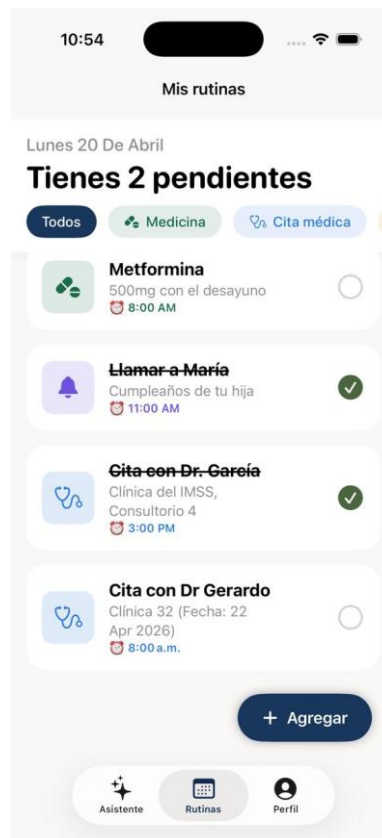


Figura 3b. Pantalla 4 — Rutinas: 1 tarea pendiente tras completar dos.

Pantalla 4 — Rutinas: Editor (EditorRutinaSheet)

Sheet modal con barra de navegación superior (Cancelar / Nueva Rutina o Editar Rutina / Guardar). En modo creación, los campos están vacíos con placeholders; en modo edición, se precargan con los valores del Recordatorio seleccionado. El Toggle «¿Es una fecha específica?» implementa la regla RN-05: activo muestra un DatePicker gráfico para seleccionar fecha; desactivado muestra los chips LMMJVSD para días de recurrencia. En ambas ramas, un DatePicker de hora en estilo .wheel permite seleccionar la hora. La sección Categoría es una lista seleccionable con ícono y checkmark. Al presionar Guardar, se valida que el título no esté vacío y se llama a `agregarRutina()` o `actualizarRutina()` según el modo.



Figura 4a. Pantalla 5 — Editor:
Formulario de nueva rutina
recurrente.

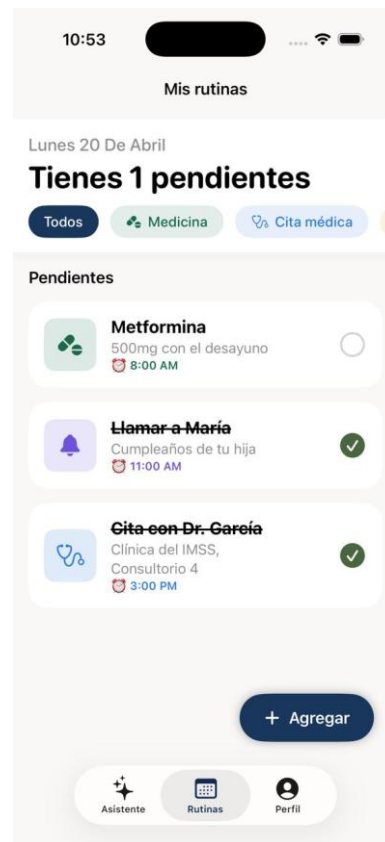


Figura 4b. Pantalla 5 — Editor:
Selector de fecha específica con
DatePicker gráfico.

Pantalla 5 — Rutinas: Selector de Días de Recurrencia

Visible dentro del EditorRutinaSheet cuando el toggle de fecha específica está inactivo. Los siete chips circulares de 44pt alternan entre seleccionado (fondo azul marino) y no seleccionado (borde azul marino, fondo transparente). El estado se gestiona con `@State` var `diasSeleccionados: Set<Int>`. El botón Guardar permanece deshabilitado mientras el Set esté vacío.

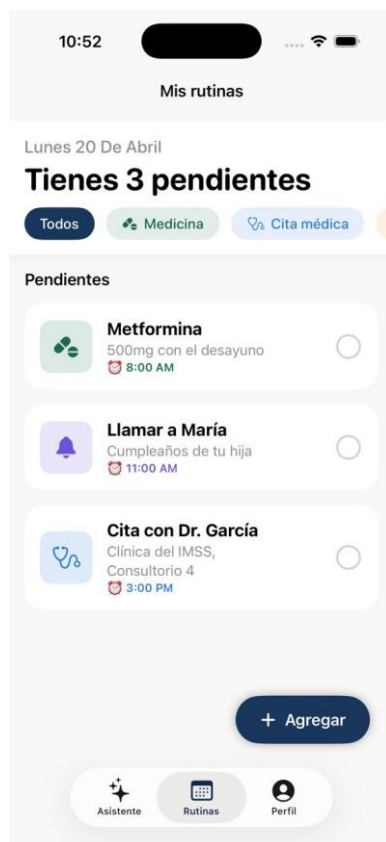


Figura 5a. Pantalla 6 — Editor: Chips de días de recurrencia con selección múltiple.

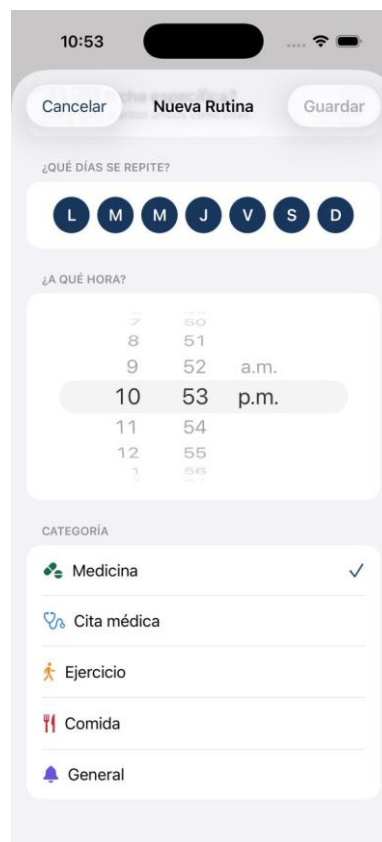


Figura 5b. Pantalla 6 — Editor: Selector de categoría (Medicina, Cita médica, Ejercicio, Comida, General).

Pantalla 6 — Perfil: Identidad y Ficha Médica

La vista de Perfil es un ScrollView con secciones separadas por espacio en lugar de líneas divisoras. La tarjeta de identidad muestra el avatar circular con la inicial del nombre en fondo azul marino, el nombre completo y la edad. La sección Ficha Médica presenta tipo de sangre (O+) y alergias (Penicilina) con iconografía contextual. Una fila «+ Agregar dato médico» actúa como call-to-action para v2.0. En el MVP todos estos datos son literales estáticos; en v2.0 provendrán del modelo persistido con SwiftData.

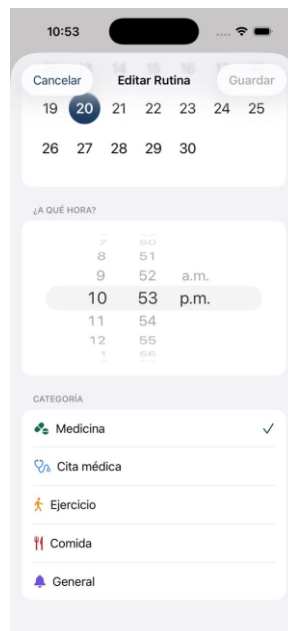


Figura 6. Pantalla 7 — Perfil: Identidad del usuario, Ficha Médica y Contacto de Alertas.

Pantalla 7 — Perfil: Contacto de Alertas y Emergencias

La sección Contacto de Alertas muestra una tarjeta con el avatar, nombre (María), relación (Hija) y badge «Dispositivo vinculado». El texto contextual —«María recibirá una notificación si olvidas una medicina»— informa al usuario de la red de seguridad sin exponer la mecánica técnica. La sección Emergencias presenta el 911 en rojo con botón de llamada directa. La Red de Apoyo lista contactos secundarios con botones de llamada individuales mediante el URL scheme tel://.

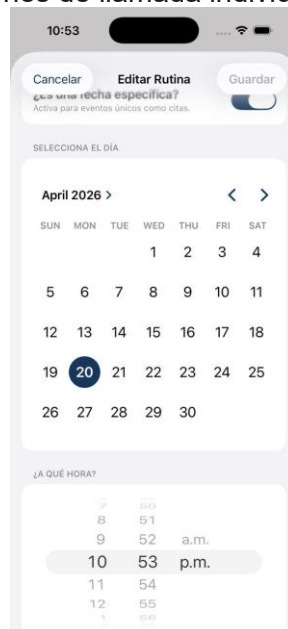


Figura 7. Pantalla 8 — Perfil: Contacto de Alertas, sección de Emergencias y Red de Apoyo.

Pantalla 8 — Perfil: Ajustes del Asistente

La sección inferior del Perfil contiene los controles de accesibilidad. El Toggle «Leer en voz alta» usa `@AppStorage("leerVozAlta")` para persistir la preferencia entre sesiones vía `UserDefaults`. Los dos Sliders de volumen y velocidad tienen iconos de extremo con SF Symbols en lugar de etiquetas numéricas, siguiendo el principio de interfaz libre de tecnicismos.



Figura 8. Pantalla 9 — Perfil: Ajustes del Asistente con toggle TTS y sliders de volumen y velocidad.

Pantalla 9 — Rutinas: Estado Completadas y Edición de Ítem

Los ítems completados aparecen con texto tachado y checkmark verde relleno. Al presionar un ítem o activar el swipe de edición, se abre el `EditorRutinaSheet` pre-rellenado. En modo edición el sheet incluye un botón «Eliminar rutina» en rojo al pie, como alternativa explícita al swipe-to-delete de la lista. Los datos mostrados siempre reflejan el estado más reciente del objeto `Recordatorio` en `GestorRutinas`, dado que SwiftUI garantiza reactividad automática sobre `@Published`.

7. Flujo de Navegación

La arquitectura de navegación de Kiali opera en dos capas. La capa principal es el `TabView`, que persiste en pantalla en todo momento y permite al usuario cambiar de módulo con un solo tap desde cualquier punto de la app. La pestaña `Asistente` (índice 0) se selecciona al iniciar la app. El color de tinte del `TabView` es `azulMarino` (`#1A3560`), que colorea el ícono de la pestaña activa.

```
TabView(selection: $tabSeleccionado) {
    AsistenteView().tabItem {
        Label("Asistente", systemImage: "sparkles")
    }.tag(0)
    RecordatoriosView().tabItem {
        Label("Rutinas", systemImage: "calendar.badge.clock")
    }.tag(1)
    PerfilView().tabItem {
        Label("Perfil", systemImage: "person.crop.circle")
    }.tag(2)
}.tint(AppColors.azulMarino)
```

La capa secundaria usa exclusivamente `sheet modals` en lugar de `NavigationStack` para las acciones de creación y edición. Esta decisión responde a que los adultos mayores reportan dificultad para comprender el botón `Back` y el concepto de jerarquía de pantallas. Los sheets tienen semántica de aparición desde abajo que el usuario distingue visualmente de la navegación lateral, y sus botones `Cancelar` y `Guardar` eliminan la ambigüedad del gesto de `swipe-down` para descartar. Al presionar `Cancelar`, el sheet se descarta sin guardar; al presionar `Guardar`, la vista llama al método correspondiente del `GestorRutinas` y luego invoca `dismiss()` del entorno de presentación.

El flujo de usuario principal —el que ocurre en un día típico de uso— parte del `Asistente`, donde la `TarjetaIntervencion` puede aparecer automáticamente. El usuario confirma con «Ya lo hice», navega a `Rutinas` para ver pendientes, filtra por `Medicina`, crea un nuevo recordatorio mediante el sheet, y finalmente revisa su `Perfil`. En ningún momento debe navegar más de un nivel de profundidad, lo que garantiza que el usuario siempre sepa dónde está y cómo volver.

8. Gestos Especiales e Interacciones

El repertorio de gestos de Kiali se limita deliberadamente a los que el usuario puede haber aprendido en otras apps nativas de iOS. Esta decisión no es una concesión al diseño simplista sino una aplicación rigurosa de los principios de HCAI: la interfaz debe adaptarse al modelo mental del usuario, no al revés. Un adulto mayor que ha usado WhatsApp o la app de Fotos de iOS ya conoce el swipe-to-delete aunque no sepa nombrarlo; Kiali reutiliza ese conocimiento previo en lugar de introducir patrones de interacción nuevos que requieran aprendizaje explícito. El gesto más relevante es el swipe-to-delete en la lista de Rutinas: deslizar horizontalmente hacia la izquierda expone el botón de eliminación con el estilo visual estándar de iOS (fondo rojo, ícono de papelera), implementado con el modificador `.onDelete` sobre el `ForEach`. En v2.0 se contempla añadir un tutorial contextual en el primer acceso a la lista para usuarios que no conozcan este gesto.

El tap en los chips de categoría actualiza `@State var categoriaSeleccionada` con una transición visual animada con `withAnimation(.easeInOut(duration:0.2))`. Los chips tienen 36pt de alto y padding horizontal de 16pt, garantizando área táctil cómoda incluso con motricidad reducida. El tap en el ícono circular de estado de cada rutina alterna entre pendiente y completada con una animación spring que proporciona retroalimentación visual sin recurrir al Taptic Engine (que requiere habilitación explícita por RN-09). El sheet del editor deshabilita el swipe-down para descartar —mediante `.interactiveDismissDisabled(true)`— cuando el usuario ha modificado algún campo, previniendo pérdidas accidentales de datos por gestos involuntarios, un riesgo real con usuarios que pueden tener temblores o imprecisión motriz.

9. Demo y Especificación Visual de Pantallas

La paleta de colores de Kiali está definida en `AppColors` dentro de `AppTheme.swift`. El azul marino `#1A3560` es el color primario de marca: se usa en el `TabView` tint, en encabezados, botones y avatares. El crema `#F8F7F4` es el fondo principal de todas las vistas, elegido por su temperatura visual cálida que reduce la fatiga ocular frente al blanco puro. El verde salvia `#4A6640` señala acciones de confirmación y la categoría Medicina. El rojo alerta `#B80000` marca elementos de urgencia: la hora de las rutinas, el número de emergencia 911 y el botón «Ya lo hice» de la `TarjetaIntervencion`. El gris `#6B7280` es el color de texto secundario y metadatos.

Las decisiones de paleta no son arbitrarias. El azul marino `#1A3560` supera el ratio WCAG AA de 4.5:1 sobre fondo crema y blanco en todos los tamaños de texto usados en la app. El rojo `#B80000` se eligió sobre el rojo puro `#FF0000` precisamente porque los adultos mayores con deuteranopía (daltonismo rojo-verde) tienen mayor facilidad para distinguir tonos de rojo oscuro sobre fondos claros. El verde salvia `#4A6640` se mantuvo en elementos secundarios y no como color semántico exclusivo para evitar dependencia del color como único canal de información, cumpliendo con el criterio 1.4.1 de WCAG. La decisión de usar crema `#F8F7F4` en lugar de blanco puro como fondo reduce el contraste máximo entre zonas blancas y de texto, aliviando la fatiga visual en sesiones prolongadas, un beneficio especialmente relevante para usuarios con cataratas incipientes, condición frecuente a partir de los 65 años.

En la pantalla del Asistente, el saludo usa SF Pro Display Bold a 28pt en `#1C1C1C`. La cuadrícula de tareas tiene celdas con borde sutil `#E5E7EB`, fondo blanco, ícono de 28pt en azul marino y etiqueta de 12pt. Cada celda mide aproximadamente 88×88pt, más del doble del mínimo requerido. El botón de micrófono tiene fondo verde salvia y ícono blanco en 44×44pt. La `TarjetaIntervencion` tiene `cornerRadius 16`, sombra de 8pt y fondo blanco. El nombre de la rutina aparece en SF Pro Display Bold 22pt; el botón «Ya lo hice» es rojo `#B80000` con texto blanco Bold 17pt y altura de 56pt.

En la pantalla de Rutinas con filtro activo, el encabezado muestra la fecha en SF Pro Text Medium 14pt gris y el contador en SF Pro Display Bold 28pt negro. El chip seleccionado tiene fondo sólido en el color de la categoría con texto blanco en Semibold 14pt; los inactivos tienen borde de 1pt en el mismo color con fondo transparente. Cada ítem de rutina tiene 72pt de alto; el texto del nombre usa SF Pro Text Semibold 16pt y la hora aparece en SF Pro Text Medium 13pt rojo. El editor de rutina muestra el calendario nativo con el día seleccionado en un círculo relleno azul marino #1A3560 con número blanco. En el Perfil, el avatar usa SF Pro Display Bold 28pt blanco sobre fondo azul marino, y el número 911 aparece en SF Pro Display Bold 20pt rojo.

10. Flujo de Datos y Gestión de Estado

La gestión de estado sigue el principio de Single Source of Truth de SwiftUI: un único objeto observable posee y administra el estado global, y las vistas suscriben sus cambios de forma reactiva. Ese objeto es `GestorRutinas`, una clase `ObservableObject` instanciada una sola vez en la raíz de la app con `@StateObject` y propagada como `environmentObject` al árbol de vistas completo.

```
@main struct Swifties2App: App {      @StateObject private var gestorRutinas =
    GestorRutinas()      var body: some Scene {      WindowGroup {
    ContentView().environmentObject(gestorRutinas)      }      }
```

La entidad central del modelo de datos es el struct `Recordatorio`, definido en `Models.swift` con las propiedades: `id`: `UUID`, `titulo`: `String`, `detalle`: `String`, `hora`: `Date`, `categoría`: `TipoRecordatorio`, `completada`: `Bool`, `esFechaEspecífica`: `Bool`, `fechaEspecífica`: `Date?` y `días`: `Set<Int>`. La propiedad `días` usa `Set` en lugar de `Array` para garantizar unicidad automática; `esFechaEspecífica` actúa como discriminador de tipo que implementa la regla RN-05. El uso de struct garantiza `value semantics`: cada modificación crea una copia nueva, haciendo los cambios predecibles y eliminando bugs de referencia compartida.

Las categorías se modelan como enum `TipoRecordatorio`: `String`, `Casalterable`, `Identifiable`, lo que permite iterar sobre todos los casos para construir los chips de filtro y el selector de categorías sin listas hardcodeadas en la capa de presentación. `GestorRutinas` expone propiedades computadas que encapsulan la lógica de negocio: `pendientesHoy` filtra por `completada == false` y verifica si la rutina corresponde al día actual (`isDateInToday` para fecha específica, comprobación de `weekday` para recurrente); `rutinaMasUrgente` retorna el `Recordatorio` pendiente con la hora más próxima al momento actual; `mostrarIntervencion` es una `@Published var Bool` que controla la visibilidad de la `TarjetaIntervencion` en el Asistente.

El patrón `ObservableObject` con `@Published` tiene implicaciones de rendimiento que deben entenderse en el contexto de Kiali. Cada vez que cualquier propiedad `@Published` de `GestorRutinas` cambia, SwiftUI invalida y re-renderiza todas las vistas que estén suscritas a ese objeto. Con un array de rutinas de tamaño pequeño —típicamente entre 5 y 20 elementos para un usuario real— este costo es despreciable. Sin embargo, en versiones futuras donde el historial de rutinas pueda crecer significativamente, será necesario dividir el estado en objetos más granulares o migrar a `SwiftData` con `@Query`, que solo re-renderiza las vistas afectadas por los cambios específicos en lugar de invalidar toda la jerarquía. Esta consideración está documentada como deuda técnica planificada y no como un bug del MVP.

El ciclo de vida de una rutina atraviesa cuatro estados. Al guardar el `EditorRutinaSheet`, `agregarRutina()` hace `append` al array `@Published` rutinas y la rutina entra en `Activa-Pendiente`. Cuando llega su hora sin confirmación, `GestorRutinas` detecta la rutina como candidata y activa `mostrarIntervencion`. El usuario responde con «Ya lo hice» o ignora la tarjeta; si confirma, `confirmarIntervencion()` llama a `completarRutina(id:)` y la rutina pasa a `Completada`. La edición en cualquier estado reabre el editor y actualiza el registro existente mediante `actualizarRutina()`. La eliminación por swipe llama a `eliminarRutinas(offsets:)`, que es el único estado terminal en el MVP dado que no existe papelera.

11. Servicios por Pantalla y Operaciones CRUD

En el MVP todas las operaciones de datos son locales y síncronas: no hay latencia de red, autenticación ni manejo de errores de conectividad. La tabla siguiente sintetiza las operaciones por pantalla.

Pantalla	Operación	Método en GestorRutinas	Efecto
Lista de Rutinas	Read	pendientesHoy, rutinasFiltradas	Recomputa lista visible reactivamente
Editor (crear)	Create	agregarRutina(_ r: Recordatorio)	Append al array @Published rutinas
Editor (editar)	Update	actualizarRutina(_ r: Recordatorio)	Reemplaza elemento en el array por id
Swipe-to-delete	Delete	eliminarRutinas(offsets: IndexSet)	Remueve del array, operación terminal
Check completada	Update	toggleCompletada(id: UUID)	Alterna completada, recalcula pendientesHoy
Tarjeta Intervención	Update	confirmarIntervencion()	Marca completada, oculta tarjeta
Perfil — Llamada	Exec	UIApplication.shared.open(URL(tel://...))	Abre la app Teléfono de iOS

Tabla 1. Operaciones CRUD y de servicio por pantalla en el MVP de Kiali.

Las integraciones de servicios externos son relevantes únicamente en versiones post-MVP. La notificación push al familiar cuidador requiere Apple Push Notification service con un backend serverless; Firebase Cloud Messaging cubre el caso de uso en su plan gratuito hasta un millón de mensajes mensuales. La autenticación del familiar se implementaría con Sign in with Apple, obligatorio en la App Store cuando se soportan métodos de autenticación de terceros. El escaneo de recetas con VisionKit es on-device y no implica costos ni transmisión de datos. La integración con HealthKit requiere el entitlement com.apple.developer.healthkit y revisión especial del equipo de App Store, extendiendo el proceso de aprobación de uno a tres días a dos o cuatro semanas.

Un aspecto crítico de las integraciones futuras es la gestión del consentimiento informado del usuario. La Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) en México clasifica los datos de salud como datos sensibles, lo que implica que su recopilación y almacenamiento requieren consentimiento expreso, informado, libre y específico del titular, no un consentimiento genérico en términos y condiciones. Esto tiene implicaciones directas en el diseño de la experiencia de configuración de Kiali: en v2.0, cuando se introduzca la sincronización con el familiar cuidador, deberá existir una pantalla de consentimiento explícita que explique en lenguaje simple qué datos se comparten, con quién y para qué propósito, con la opción de revocar ese consentimiento en cualquier momento desde la sección de Perfil.

12. Almacenamiento Local y Privacidad de Datos

El enfoque de almacenamiento de Kiali está guiado por el principio de privacidad by design, definido por el Reglamento General de Protección de Datos (GDPR) europeo y adoptado por la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) en México: los mecanismos de protección de datos deben estar integrados en la arquitectura del sistema desde su concepción, no añadidos como capa posterior. Toda la información personal y médica permanece exclusivamente en el dispositivo. Esta decisión no es únicamente técnica

sino ética, y responde a la naturaleza altamente sensible de los datos gestionados: historial médico, alergias, contactos de emergencia y patrones de comportamiento diario.

En el MVP, todos los datos viven en memoria durante la sesión activa. La única excepción es la preferencia de lectura en voz alta, almacenada con `@AppStorage("leerVozAlta")` en `UserDefaults`, mecanismo apropiado para booleanos y valores primitivos pero inadecuado para colecciones de structs. La migración a `SwiftData` en v2.0 requiere tres cambios: decorar el struct `Recordatorio` con el macro `@Model` (que convierte el tipo de struct a class para el tracking de cambios), reemplazar el array `@Published` por una `@Query` de `SwiftData` en el `GestorRutinas`, e inyectar el `ModelContext` en el punto de entrada de la app. El historial de rutinas, los datos del perfil, la ficha médica y los contactos de apoyo almacenados en v2.0 nunca deberán salir del dispositivo sin consentimiento explícito del usuario, requisito que se implementa técnicamente a través de la configuración `private` de los `CKContainer` de `CloudKit` cuando se active la sincronización familiar en v3.0. Los tokens push del familiar son credenciales de acceso y deben almacenarse en el `Keychain`, nunca en `UserDefaults`.

13. Sensores y Capacidades del Teléfono

Kiali hace uso deliberadamente restringido de los sensores y capacidades del dispositivo, siguiendo el principio de mínimo privilegio: no se solicita acceso a ningún sensor que no sea estrictamente necesario para la funcionalidad declarada. Este principio es especialmente importante para una aplicación que maneja datos médicos sensibles y cuyo usuario principal es un adulto mayor que puede no comprender completamente las implicaciones de cada permiso que otorga. Desde la perspectiva de la App Store, Apple aplica un escrutinio particularmente riguroso a las apps de salud que solicitan permisos de cámara, micrófono o `HealthKit`, requiriendo que el texto de justificación en el `Info.plist` sea específico, honesto y directamente relacionado con la funcionalidad declarada en la página de la app.

La síntesis de voz (TTS) es la capacidad más importante planificada para v2.0. `AVSpeechSynthesizer` soporta voces de alta calidad en español mexicano (es-MX) desde iOS 16, opera completamente offline y no requiere permiso del sistema. Los parámetros `rate` y `volume` del `AVSpeechUtterance` se mapean directamente a los sliders del módulo Perfil ya implementados. Para que la voz sea audible con el teléfono en modo silencioso, es necesario configurar `AVAudioSession` con la categoría `.playback` antes de iniciar la síntesis; esta configuración debe manejarse en el `ViewModel` y no en la vista.

El reconocimiento de voz (STT) mediante `SFSpeechRecognizer` y `AVAudioEngine` está planificado para v2.0 y requiere dos permisos en el `Info.plist`: `NSMicrophoneUsageDescription` y `NSSpeechRecognitionUsageDescription`. iOS 16 introdujo reconocimiento on-device para español, lo que significa que en versiones modernas del sistema la transcripción ocurre sin conectividad, preservando la privacidad del usuario. En versiones anteriores de iOS, el audio se procesa en servidores de Apple bajo su política de privacidad. El escaneo de recetas médicas en v3.0 utiliza `DataScannerViewController` de `VisionKit`, que provee OCR on-device en tiempo real mediante la cámara trasera, requiriendo el permiso `NSCameraUsageDescription`. El `Taptic Engine`, accesible mediante `UIImpactFeedbackGenerator`, está fuera del MVP por la regla RN-09, pero en v2.0 se contempla usar haptics de baja intensidad (`intensity: 0.5`) al confirmar rutinas, dado que la retroalimentación háptica mejora significativamente la percepción de éxito en usuarios con baja agudeza visual.

14. Lenguajes y Herramientas

14.1 Frontend: Swift y SwiftUI

Kiali está escrita íntegramente en Swift 5.9, el lenguaje de programación oficial de Apple para el desarrollo de aplicaciones en todo su ecosistema. Swift combina seguridad de tipos estática, inferencia de tipos, manejo explícito de opcionales y value semantics mediante structs, características que en conjunto hacen que una categoría entera de bugs comunes en lenguajes como Objective-C —punteros nulos no gestionados, mutación inesperada de estado compartido, desbordamientos de memoria— sean imposibles o detectables en tiempo de compilación. Para una aplicación desarrollada en 8 horas con un equipo reducido, este nivel de seguridad del lenguaje fue un factor determinante en la velocidad de desarrollo: el compilador de Xcode interceptó errores que en otros entornos solo se descubrirían en pruebas manuales.

SwiftUI es el framework declarativo de construcción de interfaces de usuario introducido por Apple en 2019 y que en la versión utilizada para Kiali (SwiftUI 4, iOS 16+) ya exhibe la madurez suficiente para producción. Su modelo de programación reactivo —las vistas son funciones puras del estado— elimina la categoría de bugs de sincronización entre estado y UI que afligía al patrón MVC con UIKit, donde el desarrollador debía coordinar manualmente cuándo actualizar cada elemento de la interfaz. En SwiftUI, esta coordinación es automática: cualquier cambio en una propiedad `@State` o `@Published` desencadena automáticamente la re-evaluación de las vistas que dependen de ese valor. Para Kiali, esto significa que añadir una rutina desde el `EditorRoutineSheet` se refleja instantáneamente en la lista de `RecordatoriosView` sin una sola línea de código de actualización explícita.

La elección de SwiftUI sobre UIKit fue inmediata y no requirió deliberación. UIKit, el framework imperativo que precedió a SwiftUI, habría requerido significativamente más código boilerplate para construir las mismas interfaces —especialmente el `EditorRoutineSheet` con sus controles condicionales y el `TabView` con estado compartido— y habría sido incompatible con la velocidad de iteración del hackathon. SwiftUI permite prototipar, ajustar y compilar en segundos lo que en UIKit tomaría minutos.

14.2 Backend: Ausencia Deliberada y Justificación

Kiali no tiene backend en el MVP, y esta es una decisión de arquitectura positiva, no una omisión. Toda la lógica de la aplicación reside en el cliente iOS. Esta elección tiene cinco consecuencias directas: elimina la latencia de red en todas las operaciones del usuario, garantiza funcionamiento offline completo (RNF-10, RN-01), evita costos de infraestructura de servidor durante la fase de validación, elimina la superficie de ataque de seguridad que representa un servidor con datos médicos de usuarios, y simplifica el stack tecnológico al punto en que dos desarrolladores pueden mantener el sistema completo sin especialización en backend.

La ausencia de backend en el MVP no significa que la arquitectura no esté preparada para él. `GestorRutinas` está diseñado como un servicio de datos con una interfaz pública clara (métodos CRUD explícitos, propiedades computadas) que puede refactorizarse para delegar operaciones a un repositorio remoto sin que las vistas tengan que cambiar. En términos del patrón Repository, las vistas interactúan con `GestorRutinas` como si fuera una fuente de datos abstracta; en v2.0, `GestorRutinas` puede convertirse en una fachada que coordina entre `SwiftData` local y `CloudKit` remoto, y las vistas no notarán la diferencia.

14.3 Herramientas de Desarrollo

El entorno de trabajo durante el hackathon combinó Xcode 16, la terminal de macOS y el navegador. Xcode 16 es el IDE de Apple y el único que permite compilar y firmar aplicaciones

iOS para distribución. Sus herramientas de diagnóstico —el Memory Graph Debugger, el View Hierarchy Debugger y el CPU Performance Report en Instruments— son fundamentales para identificar retains cycles, elementos de interfaz solapados y consumo excesivo de procesador, problemas que no son evidentes durante el desarrollo normal pero que emergen bajo carga real. El View Hierarchy Debugger fue usado durante el hackathon para verificar que la TarjetaIntervencion se superponía correctamente como overlay sin bloquear los gestos de la capa de contenido subyacente del Asistente. Fue usado en su versión 16 por ambos integrantes del equipo durante el hackathon. El simulador de Xcode permitió probar la aplicación en múltiples modelos de iPhone sin dispositivos físicos, aunque con la limitación ya documentada de que las notificaciones locales no se disparan en el simulador. Git y GitHub fueron las herramientas de control de versiones y colaboración. El repositorio en github.com/obeedt/AppSwift sirvió como punto de integración entre los dos desarrolladores y como medio de entrega de todos los artefactos del proyecto. No se usaron herramientas de gestión de dependencias externas como Swift Package Manager o CocoaPods dado que Kiali no tiene dependencias de terceros; todos los frameworks utilizados son parte del SDK de Apple incluido en Xcode.

15. Arquitectura y Patrones de Diseño

15.1 MVVM en Kiali: Implementación Concreta

El patrón Model-View-ViewModel (MVVM) es la arquitectura recomendada por Apple para aplicaciones SwiftUI, y Kiali lo implementa en su forma canónica. La capa Model está representada por el struct Recordatorio y el enum TipoRecordatorio, definidos en Models.swift. Son tipos de datos puros sin lógica de presentación ni dependencias de UIKit o SwiftUI. La capa ViewModel está representada por GestorRutinas, una clase ObservableObject que centraliza el estado global y expone operaciones de negocio como métodos con nombres semánticamente claros: agregarRutina, actualizarRutina, toggleCompletada, confirmarIntervencion. La capa View está representada por AsistenteView, RecordatoriosView, PerfilView y sus componentes auxiliares (TarjetaIntervencion, RowRutina, EditorRutinaSheet), que son estructuras SwiftUI que solo consumen estado y disparan acciones —no contienen lógica de negocio ni manipulan el modelo directamente.

La separación entre capas tiene una ventaja práctica inmediata durante el hackathon: los dos integrantes del equipo pudieron trabajar simultáneamente en módulos distintos sin conflictos de código, porque las vistas y el ViewModel estaban en archivos separados con interfaces bien definidas. Torres Pimentel podía modificar la lógica de GestorRutinas mientras Sánchez Medina ajustaba el diseño de EditorRutinaSheet, y el único punto de integración era la firma pública de los métodos del ViewModel, que permanecía estable.

Una decisión arquitectónica relevante fue usar @StateObject para instanciar GestorRutinas en la raíz de la app y @EnvironmentObject para accederlo en las vistas hijas, en lugar de @ObservedObject con inyección manual a través de inicializadores. EnvironmentObject elimina el prop drilling —el antipatrón de pasar el mismo objeto por cuatro o cinco niveles de la jerarquía de vistas solo para que la vista más anidada lo use—, que habría sido especialmente tedioso en una app con TabView y sheets modales donde el GestorRutinas es necesario en múltiples ramas del árbol de vistas simultáneamente.

```
// Acceso al ViewModel desde cualquier vista hija struct RecordatoriosView: View {
@EnvironmentObject var gestorRutinas: GestorRutinas // No se necesita
init(gestorRutinas:) explicito }
```

15.2 Patrones Adicionales Aplicados

Además de MVVM, Kiali aplica varios patrones de diseño de software de manera implícita. El patrón Observer está directamente implementado por el mecanismo `@Published / @EnvironmentObject` de Combine y SwiftUI: las vistas son observadoras del GestorRutinas y se actualizan automáticamente al cambiar el estado. El patrón Command se aplica en los botones de acción de las vistas, que encapsulan la intención del usuario en llamadas a métodos del ViewModel con nombres que describen la acción semánticamente (confirmarIntervencion, no setMostrarIntervencion(false)). El patrón Factory se usa implícitamente en la función de inicialización del GestorRutinas, que construye los datos de ejemplo del hackathon mediante un método privado `poblarDatosDemo()`. El patrón Single Source of Truth, fundamental en la filosofía de SwiftUI, se garantiza a través del ViewModel único: ninguna vista mantiene una copia local del estado que pueda desincronizarse con el estado global.

16. Administración del Proyecto

16.1 Metodología: Micro-Sprints de Hackathon

El contexto de un hackathon de 8 horas impone una metodología radicalmente diferente a la de un proyecto de software convencional. No hay espacio para ceremonias de Scrum, sprints de dos semanas ni planificación de backlog exhaustiva. El equipo Swifties adoptó un enfoque de micro-sprints de 90 minutos, donde cada sprint tenía un objetivo de entrega vertical —no una lista de tareas— y terminaba con código compilable y demostrable. Esta práctica, informada por los principios de desarrollo ágil pero adaptada a la escala del hackathon, garantizó que en cualquier momento del día el equipo tuviera una versión funcional de la app que podía presentarse si el tiempo se agotaba inesperadamente.

La distribución aproximada de los micro-sprints fue la siguiente. El primer sprint de 90 minutos se dedicó a la definición del problema, ideación del nombre Kiali, estructuración de los tres módulos en el TabView, creación de Models.swift con el struct Recordatorio y configuración del repositorio en GitHub. El segundo sprint estableció el GestorRutinas con datos de ejemplo y la vista completa de RecordatoriosView incluyendo la lógica de filtrado por categoría. El tercero construyó el EditorRutinaSheet completo con el toggle de tipo de rutina, el DatePicker gráfico y los chips de días. El cuarto sprint se focalizó en el módulo Asistente: saludo contextual, cuadrícula de tareas frecuentes y la lógica de la TarjetaIntervencion. El quinto completó el módulo Perfil y los ajustes del asistente. El sexto y último fue de integración, pulido visual, aplicación consistente de AppColors y AppTheme, y preparación de la demo con datos representativos.

16.2 Gestión de Riesgos Realizada

La gestión del tiempo durante el hackathon exigió disciplina activa. El equipo estableció checkpoints explícitos al final de cada micro-sprint: si el objetivo del sprint no estaba completo al 80% al llegar al checkpoint, el scope se reducía en lugar de extender el tiempo. Esta regla se aplicó en el sprint tres, donde el EditorRutinaSheet completó el 90% del diseño previsto pero la validación del campo de título y el manejo del botón Guardar deshabilitado con formulario vacío quedaron para el sprint de integración. La alternativa habitual en equipos sin esta disciplina —extender el sprint y recortar el siguiente— genera una deuda de tiempo que se acumula y que en un hackathon de 8 horas resulta en una presentación incompleta o caótica.

El principal riesgo técnico materializado durante el hackathon fue la contaminación de comillas tipográficas al copiar fragmentos de código desde el asistente de IA utilizado para consultas técnicas. El autocorrector de macOS reemplazó las comillas rectas ASCII requeridas por Swift (") con comillas curvas tipográficas («»), generando errores de compilación del tipo Unicode curly

quote found en Xcode que no son inmediatamente obvios para el desarrollador. La solución adoptada fue descargar los archivos de código directamente al sistema de archivos en lugar de copiar texto desde el navegador, eliminando el paso por el autocorrector. Esta lección operativa, aunque menor en apariencia, ilustra un principio general de ingeniería: los errores de entorno y configuración consumen tiempo desproporcionado en contextos de alta presión y deben identificarse y mitigarse temprano.

Un segundo riesgo gestionado fue el de scope creep: la tentación de añadir funcionalidades adicionales a medida que los módulos base se completaban antes de lo previsto. El equipo mantuvo disciplina de alcance descartando explícitamente tres ideas generadas durante el hackathon —un módulo de chat familiar, una pantalla de estadísticas de cumplimiento de rutinas y un modo de pantalla grande para televisores via AirPlay— no por falta de capacidad técnica sino por coherencia con el MVP definido y por el riesgo de introducir bugs de integración en las últimas horas del evento.

17. Uso de Inteligencia Artificial en el Ciclo de Desarrollo

Kiali es, paradójicamente, una aplicación sobre IA centrada en el humano que fue desarrollada con el apoyo de herramientas de IA generativa. Esta dualidad merece documentarse con honestidad técnica, diferenciando los roles que la IA desempeñó en el proceso de desarrollo del rol que desempeña dentro de la aplicación misma.

17.1 IA como Herramienta de Desarrollo

Durante el hackathon, el equipo utilizó asistentes de IA generativa —específicamente Claude de Anthropic— como herramienta de consulta técnica y generación de código base. Los usos concretos incluyeron: generación de la estructura inicial del struct Recordatorio y el enum TipoRecordatorio a partir de una descripción verbal de los requerimientos; sugerencias sobre la implementación del toggle de tipo de rutina con su lógica condicional en el EditorRutinaSheet; consultas sobre la sintaxis correcta de @EnvironmentObject y el patrón de inyección de dependencias en SwiftUI; y generación del código de la animación SubtleLogoAnimation con withAnimation. En todos los casos, el código generado fue revisado, adaptado y en varios casos reescrito por el equipo antes de integrarse al repositorio.

17.2 IA como Funcionalidad de la Aplicación

Dentro de la aplicación, la IA se implementa en el MVP como lógica de negocio determinista que simula comportamiento inteligente. La TarjetaIntervencion no usa un modelo de machine learning sino una evaluación condicional sobre el estado de las rutinas. La guía de pasos del Asistente no genera texto dinámicamente sino que recupera respuestas predefinidas de un diccionario. Esta honestidad de implementación no invalida la propuesta de valor de HCAI: para el usuario adulto mayor, el efecto es indistinguible de una IA sofisticada, y la experiencia cumple con el principio de que la tecnología debe cuidar sin que lo notes. El uso productivo de asistentes de IA generativa en el hackathon requirió una habilidad que el currículum de ingeniería tradicional rara vez enseña explícitamente: el prompt engineering, es decir, la capacidad de formular consultas precisas que produzcan respuestas útiles. Preguntar a un asistente de IA «cómo hago un toggle en SwiftUI» produce una respuesta genérica. Preguntar «en SwiftUI, cómo implemento un Toggle que controle una propiedad @Published en un ObservableObject y persista la preferencia entre sesiones usando @AppStorage, considerando que el ViewModel está inyectado como @EnvironmentObject» produce código directamente aplicable al contexto de Kiali. Esta capacidad de especificar el contexto técnico relevante es una habilidad de ingeniería, no una habilidad de usuario de IA, y su desarrollo fue uno de los aprendizajes transversales del hackathon para el equipo.

La hoja de ruta de v3.0 y v4.0 contempla la migración hacia IA genuina mediante Core ML para personalización de intervenciones, y la integración con App Intents para que Siri pueda interpretar comandos en lenguaje natural como «Kiali, ya tomé mi pastilla» y actualizar el estado de la rutina correspondiente. Este es el momento en que la IA de Kiali dejará de ser simulada y se convertirá en inferencia real sobre datos del usuario, siempre on-device y con privacidad preservada.

18. Frameworks y Kits Utilizados

Kiali hace uso exclusivo de frameworks del SDK oficial de Apple incluido en Xcode 16, sin dependencias de terceros. Esta decisión garantiza estabilidad a largo plazo —Apple mantiene compatibilidad hacia atrás con más rigor que las librerías open source—, elimina la complejidad de gestión de paquetes con Swift Package Manager o CocoaPods, y reduce la superficie de riesgo de seguridad que introduce cualquier código externo no auditado.

SwiftUI

Framework principal de construcción de interfaces de usuario, usado para todas las vistas de la aplicación. Su modelo declarativo y reactivo es el fundamento sobre el que se implementa la arquitectura MVVM de Kiali. Versión mínima requerida: SwiftUI disponible desde iOS 13, con las funcionalidades específicas usadas en Kiali (DatePicker gráfico, .interactiveDismissDisabled, chips condicionales) disponibles desde iOS 16.

Combine

Framework de programación reactiva de Apple que alimenta el mecanismo de @Published y ObservableObject. Aunque Kiali no usa operadores de Combine explícitamente en el código de las vistas, la maquinaria subyacente que propaga los cambios de GestorRutinas hacia las vistas suscritas está implementada por Combine internamente. En versiones futuras, Combine podría usarse explícitamente para implementar debouncing en el campo de búsqueda del Asistente (evitar búsquedas con cada tecla) mediante el operador .debounce(for:scheduler:).

Foundation

Framework base que provee las estructuras de datos fundamentales usadas en Kiali: UUID para identificadores únicos de rutinas, Date para almacenar horas programadas, Calendar para calcular si una rutina corresponde al día actual y para obtener el componente de hora del día para el saludo contextual, y Set<Int> para el conjunto de días de recurrencia. Foundation también provee DateFormatter para los formatos de fecha que aparecen en el encabezado de RecordatoriosView.

SF Symbols

Biblioteca de más de 6,000 íconos vectoriales integrados en iOS, accesibles mediante Image(systemName:) en SwiftUI. Kiali usa SF Symbols extensivamente para la iconografía de la cuadrícula del Asistente, los indicadores de categoría en la lista de Rutinas, los botones de acción en el Perfil y los íconos de la barra TabView. Los SF Symbols se escalan automáticamente con Dynamic Type, mantienen proporciones correctas en todos los tamaños de pantalla y tienen variantes de peso (regular, semibold, bold) que Kiali usa para mantener consistencia visual.

AVFoundation (planificado v2.0)

Framework multimedia de Apple que provee AVSpeechSynthesizer para síntesis de voz TTS. La infraestructura de configuración en el módulo Perfil —toggle de activación, sliders de volumen y velocidad— ya está implementada en el MVP en anticipación a esta integración. AVSpeechSynthesizer no requiere permisos del sistema y opera completamente offline, lo que lo convierte en el primer framework adicional a integrar en v2.0.

UserNotifications (planificado v2.0)

Framework de notificaciones locales y push de iOS. Su integración en v2.0 para las alertas de rutinas es la funcionalidad de mayor impacto para la experiencia real del usuario. El patrón de integración consiste en registrar una `UNCalendarNotificationTrigger` al crear cada rutina y cancelarla

con `UNUserNotificationCenter.current().removePendingNotificationRequests(withIdentifiers:)` al marcar la rutina como completada o eliminarla.

SwiftData (planificado v2.0)

ORM oficial de Apple introducido en iOS 17 que reemplaza a `CoreData` con una API declarativa compatible con `SwiftUI`. La migración del struct `Recordatorio` a `@Model` es el cambio central de v2.0 y desbloquea la persistencia real entre sesiones, condición bloqueante para la distribución en App Store.

19. Permisos y Requisitos para la App Store

19.1 Estado del MVP: Cero Permisos Requeridos

En su versión MVP, Kiali no solicita ningún permiso del sistema operativo. No accede a la cámara, al micrófono, a los contactos del teléfono, a la ubicación, a las notificaciones ni a ningún sensor del dispositivo. Esta característica es un punto positivo de cara a la revisión de la App Store: Apple aplica escrutinio especial a las apps que solicitan permisos sensibles, y una app que no los solicita pasa la revisión técnica inicial sin fricciones. Para el MVP, el único requisito de `Info.plist` es la declaración del lenguaje base (`CFBundleDevelopmentRegion: es`) y la restricción de orientación `Portrait`.

19.2 Permisos Requeridos en Versiones Futuras

La transición a v2.0 requerirá añadir al `Info.plist` la clave `NSUserNotificationsUsageDescription` con una descripción en español claro y orientado al usuario: por ejemplo, «Kiali usa notificaciones para recordarte tomar tus medicamentos y asistir a tus citas médicas a la hora que programaste.» El texto debe ser específico y honesto; Apple rechaza descripciones genéricas del tipo «Para enviar notificaciones importantes.» El permiso de notificaciones se solicita mediante `UNUserNotificationCenter.current().requestAuthorization(options:)` en un momento contextualmente apropiado —no al abrir la app por primera vez, sino al crear la primera rutina, cuando el usuario ya entiende el valor de la notificación.

La integración del reconocimiento de voz en v2.0 requerirá añadir tanto `NSMicrophoneUsageDescription` como `NSSpeechRecognitionUsageDescription`. Para el escaneo de recetas médicas en v3.0, se requerirá `NSCameraUsageDescription`. La integración con `HealthKit` en v4.0 requiere el entitlement especial `com.apple.developer.healthkit`, que debe declararse en el archivo de capacidades del proyecto en Xcode y está sujeto a revisión adicional por parte del equipo de App Store específico para aplicaciones de salud. Este proceso puede extender el tiempo de aprobación inicial de uno a tres días a dos a cuatro semanas y puede requerir que el desarrollador proporcione documentación adicional sobre el propósito de acceso a los datos de salud.

19.3 Requisitos de Contenido para la App Store

Para publicar en la App Store, Kiali requerirá una Privacy Policy pública, el Nutrition Label de privacidad en App Store Connect —declarando `Data Not Collected` para el MVP—, y capturas de pantalla que muestren el flujo real en las resoluciones de los modelos soportados. Desde 2024, Apple permite declarar características de accesibilidad en la página de la app (`Dynamic Type`, `VoiceOver`), lo que representa una oportunidad de diferenciación para Kiali frente al familiar que busca una app para su padre.

20. Equipo y Roles

20.1 Composición del Equipo Swifties

El equipo Swifties estuvo integrado por José Santiago Sánchez Medina y Obed Torres Pimentel. Ambos integrantes participaron activamente en todas las fases del proyecto — ideación, diseño, desarrollo y presentación— asumiendo roles primarios diferenciados que maximizaron la velocidad de desarrollo durante las 8 horas del hackathon.

20.2 Distribución de Roles

Sánchez Medina asumió el rol primario de Arquitecto de Software y Technical Lead, responsable de la definición y mantenimiento de la arquitectura MVVM, la creación de Models.swift y GestorRutinas, la gestión del repositorio en GitHub y las decisiones de estructura de código. También lideró el desarrollo del módulo Asistente, incluyendo la lógica de la TarjetaIntervencion y la animación SubtleLogoAnimation.

Torres Pimentel asumió el rol primario de Desarrollador UI/UX y Frontend Lead, responsable de la definición de la paleta de colores y los tokens de diseño en AppTheme.swift y AppColors, el desarrollo de RecordatoriosView y EditorRutinaSheet con toda su lógica condicional, y el módulo PerfilView con sus secciones de ficha médica, contactos y ajustes. Torres Pimentel también preparó los datos de demostración y coordinó la presentación final ante el jurado.

Esta distribución de roles no fue rígida: ambos integrantes revisaron y contribuyeron al código del otro, especialmente en la fase de integración y pulido del último micro-sprint. La revisión cruzada de código, aunque informal en el contexto del hackathon, fue suficiente para identificar inconsistencias de estilo visual y bugs de estado antes de la presentación.

20.3 Equipo Requerido para Producción

Una práctica de gestión del conocimiento que el equipo mantuvo durante el hackathon fue la documentación inline del código: los métodos de GestorRutinas tienen comentarios que describen su propósito y las decisiones de diseño no obvias, como el uso de Set<Int> para días en lugar de Array, o la razón por la que mostrarIntervencion es una propiedad @Published del ViewModel en lugar de un @State local de la vista. Esta documentación, aunque mínima, será crítica cuando el equipo retome el proyecto semanas o meses después del hackathon, o cuando se incorpore un nuevo desarrollador que deba entender las decisiones tomadas bajo la presión de las 8 horas.

El escalamiento de Kiali de prototipo de hackathon a producto distributable en la App Store requeriría un equipo más amplio. Para v2.0, el equipo mínimo viable incluye los dos desarrolladores iOS actuales para la implementación de SwiftUI y la integración de frameworks, más un diseñador UX especializado en accesibilidad para conducir pruebas de usabilidad con usuarios reales de 65+ años —fase crítica que el hackathon no pudo cubrir— y un QA tester para validación en dispositivos físicos. Para v3.0, la introducción de CloudKit y el backend para notificaciones push requeriría incorporar un ingeniero de backend o utilizar Firebase como plataforma serverless que permita al equipo iOS mantener el ownership del backend sin especialización en infraestructura de servidor.

21. Estimaciones de Tiempo, Costo y Mantenimiento

21.1 Estimación de Desarrollo Post-MVP

Las estimaciones que siguen corresponden a un equipo de dos desarrolladores iOS de nivel semi-senior trabajando en jornadas de 6 horas diarias de desarrollo efectivo, con ciclos de revisión de código y pruebas incluidos. No incluyen tiempo de diseño UX o pruebas con usuarios reales, que deberían añadirse según el alcance de cada versión.

La versión 2.0, que añade SwiftData, notificaciones locales, TTS funcional y llamadas directas, se estima en 3 a 4 semanas de desarrollo. El mayor costo de tiempo es la migración del modelo de datos a SwiftData —incluyendo la conversión de struct a class y la validación de que el comportamiento reactivo de las vistas se mantiene— y la gestión correcta del ciclo de vida de las notificaciones, que requiere pruebas exhaustivas en dispositivo físico para cubrir todos los edge cases de creación, edición y eliminación de rutinas. La versión 3.0, que incorpora CloudKit, VisionKit y el dashboard familiar, se estima en 8 a 12 semanas. CloudKit tiene una curva de aprendizaje significativa, especialmente en la gestión de conflictos de sincronización y en el diseño del esquema de datos compartidos entre el dispositivo del adulto mayor y el del familiar. La versión 4.0 con App Intents para Siri, watchOS companion app y Core ML se estima en 16 a 20 semanas adicionales.

21.2 Estimación de Costos de Desarrollo

El costo de desarrollo está dominado por el costo del tiempo de los desarrolladores. Considerando una tarifa de mercado de 35,000 a 50,000 MXN mensuales para un desarrollador iOS semi-senior en México en 2026, el costo de desarrollar v2.0 oscila entre 70,000 y 133,000 MXN para el equipo de dos personas. Los costos de infraestructura en el MVP son cero, dado que no hay servidor. En v2.0, CloudKit está incluido en el Apple Developer Program sin costo adicional dentro de los límites de uso gratuito (que cubren holgadamente una base de usuarios de miles de personas). La cuota del Apple Developer Program es de \$99 USD anuales, requerida para publicar en la App Store. Firebase en su plan gratuito Spark cubre los requerimientos de v2.0 para mensajería push sin costo. El único costo fijo garantizado es la cuota anual de Apple.

21.3 Estimación de Costos de Mantenimiento

El mantenimiento de una aplicación iOS tiene tres fuentes de costo recurrente. Las actualizaciones de compatibilidad con nuevas versiones de iOS representan el costo de mantenimiento más predecible: Apple lanza una versión mayor de iOS cada septiembre, y los cambios en las APIs de SwiftUI en versiones recientes han requerido ajustes en código en aproximadamente el 20% de las aplicaciones. Para Kiali, dado el uso exclusivo de APIs estándar sin hacks ni workarounds, el costo de compatibilidad se estima en 2 a 4 días de trabajo por versión mayor de iOS. Las correcciones de bugs reportados por usuarios representan un costo variable dependiente de la base de usuarios activos y la calidad del testing inicial. Las actualizaciones funcionales, si el producto se desarrolla de acuerdo con la hoja de ruta descrita, representan el mayor costo de mantenimiento pero también el de mayor valor estratégico.

21.4 Modelo de Sostenibilidad Económica

El modelo de negocio freemium familiar descrito en el reporte académico complementario define dos niveles: una versión gratuita con funcionalidad completa para el adulto mayor y una suscripción premium Kiali Plus dirigida al familiar cuidador a un precio estimado de 79 MXN mensuales o 699 MXN anuales. Con una base de 1,000 suscriptores activos —meta conservadora para un nicho especializado en el primer año— los ingresos brutos anuales serían de aproximadamente 699,000 MXN, suficientes para cubrir el costo de mantenimiento y un desarrollador a tiempo parcial para la hoja de ruta de v3.0. Apple retiene el 30% de las suscripciones en la App Store el primer año y el 15% a partir del segundo, lo que debe considerarse en el modelo financiero.

22. Referencias

Las siguientes referencias bibliográficas y técnicas sustentan los datos, argumentos y decisiones de diseño documentados en este reporte. El formato de citación sigue la norma APA 7.^a edición.

- Apple Inc. (2023). Human Interface Guidelines. Apple Developer Documentation. <https://developer.apple.com/design/human-interface-guidelines/>
- Apple Inc. (2024d). UserNotifications. Apple Developer Documentation. <https://developer.apple.com/documentation/usernotifications>
- Comisión Económica para América Latina y el Caribe [CEPAL]. (2023). Envejecimiento en América Latina y el Caribe: inclusión y derechos de las personas mayores. CEPAL. <https://www.cepal.org>
- Fisher, B., y Tronto, J. (1999). Toward a feminist theory of caring. En E. K. Abel y M. K. Nelson (Eds.), *Circles of care: Work and identity in women's lives* (pp. 35–62). SUNY Press.
- Instituto Nacional de Estadística y Geografía [INEGI]. (2023). Encuesta Nacional sobre Disponibilidad y Uso de Tecnologías de la Información en los Hogares (ENDUTIH) 2022 (Comunicado de prensa núm. 367/23). INEGI. https://www.inegi.org.mx/contenidos/saladeprensa/boletines/2023/ENDUTIH/ENDUTIH_22.pdf
- MICARE Chile. (2025, 28 de abril). Uso de la tecnología en personas mayores con discapacidad intelectual. <https://www.micare.cl/2025/04/28/uso-de-la-tecnologia-en-personas-mayores-con-discapacidad-intelectual/>
- Organización Mundial de la Salud [OMS]. (2015). Informe mundial sobre el envejecimiento y la salud. OMS. https://iris.who.int/bitstream/handle/10665/186466/9789240694873_spa.pdf
- Organización Mundial de la Salud [OMS]. (2022). Envejecimiento y salud. <https://www.who.int/es/news-room/fact-sheets/detail/ageing-and-health>
- Pinazo-Hernandis, S. (2024). Las personas mayores, las tecnologías y los cuidados. *Avances y retos. SCIO. Revista de Filosofía*, (26), 73–100. https://doi.org/10.46583/scio_2024.26.1152
- Pinazo-Hernandis, S., Blanco-Molina, M., y Ortega-Moreno, R. (2022). Aging in place: connections, relationships, social participation and social support in the face of crisis situations. *International Journal of Environmental Research and Public Health*, 19(24), 16623. <https://doi.org/10.3390/ijerph192416623>
- Secretaría de Gobernación de México. (2010). Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP). Diario Oficial de la Federación. https://www.dof.gob.mx/nota_detalle.php?codigo=5150631
- World Wide Web Consortium [W3C]. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>
- Yáñez Soto, M., Flores Martínez, R. M., y Mendoza Cárdenas, E. (2026). La brecha digital en las personas adultas mayores en México: retos para la inclusión tecnológica y el papel del trabajo social. *Revista ACANITS Redes Temáticas en Trabajo Social*, 5(8), 119–137. <https://doi.org/10.62621/tf1tv897>